

Which Tools May Your AI Agent Use?

MCP in Practice — finding use cases and vetting servers

David Goll · AI Service Centre Berlin-Brandenburg (HPI) · *date TBD*

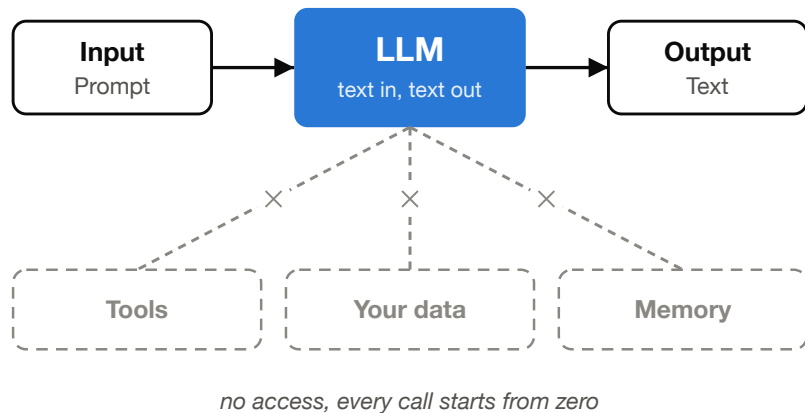
Gefördert durch:



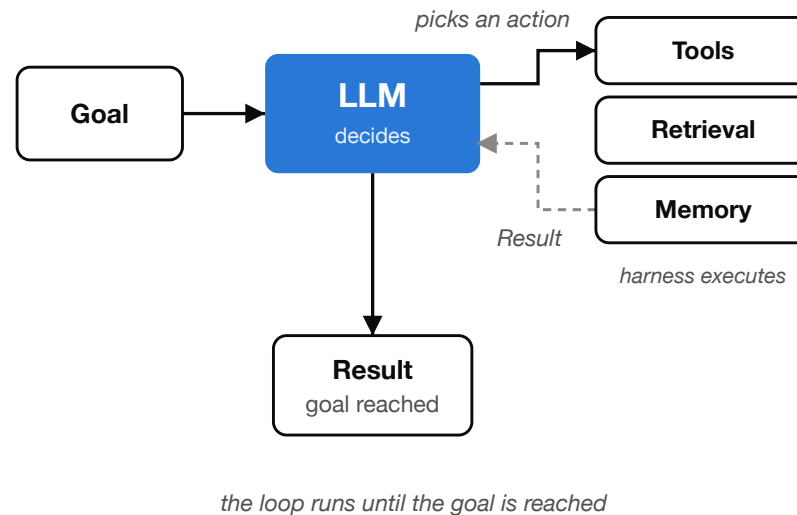
Today: 15 min concepts → 10 min live example → 20 min your own use case → 15 min discussion

What is an agent?

The LLM on its own



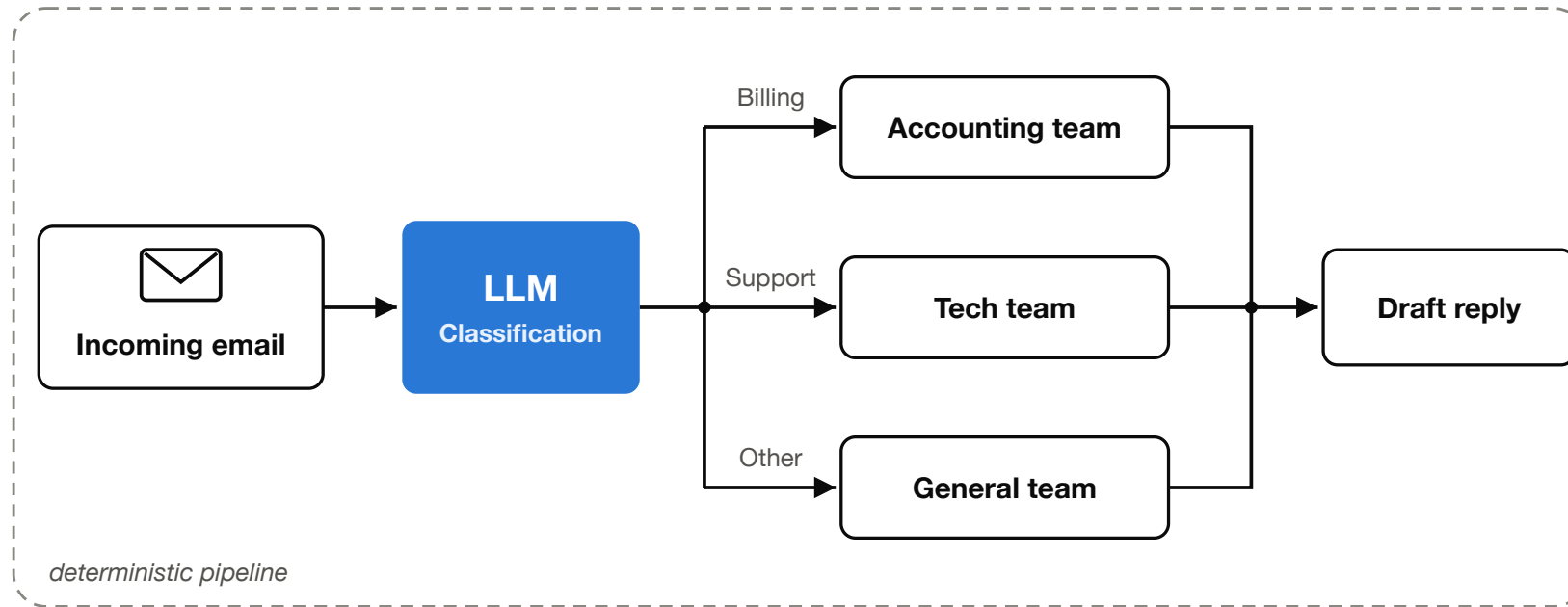
The agent



"An LLM agent runs tools in a loop to achieve a goal." — Simon Willison

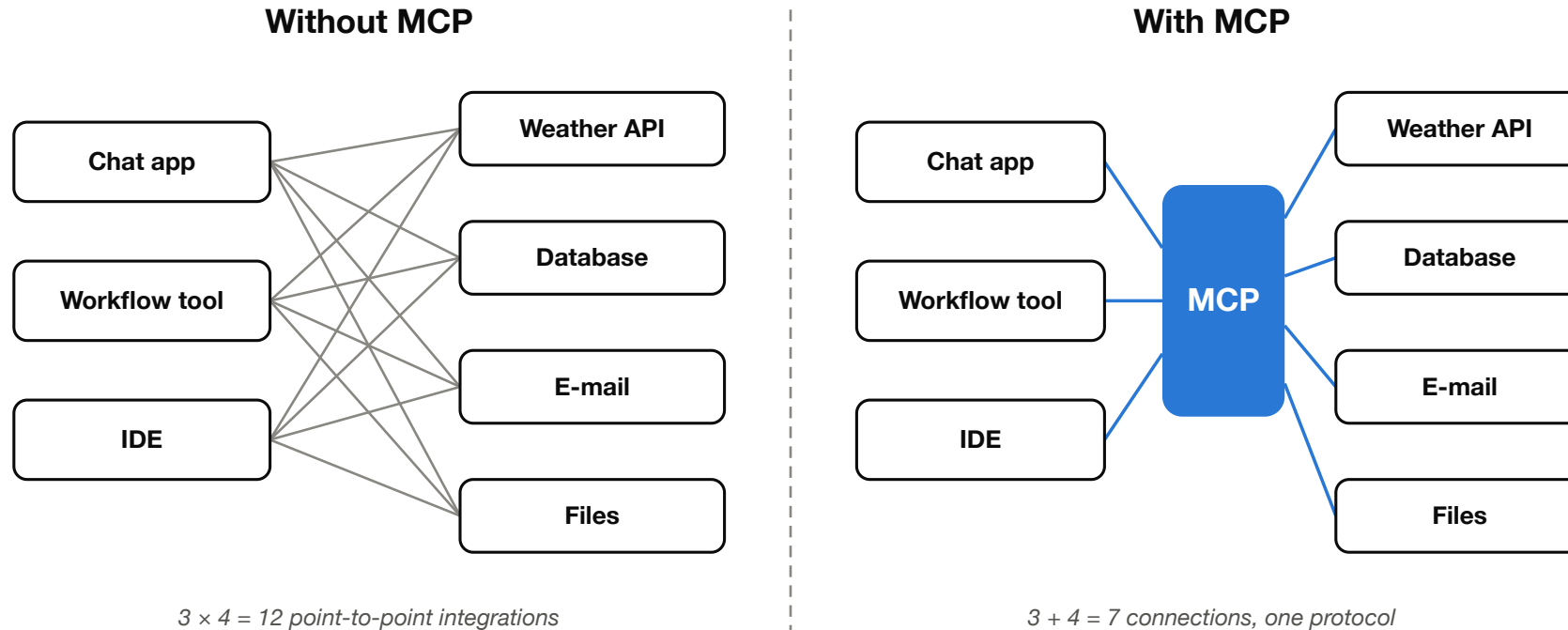
The model asks for an action · the software executes it · the result flows back · repeat until done

A concrete example: email triage



Only the blue node is the AI model — everything else is ordinary software and *tools it can call*

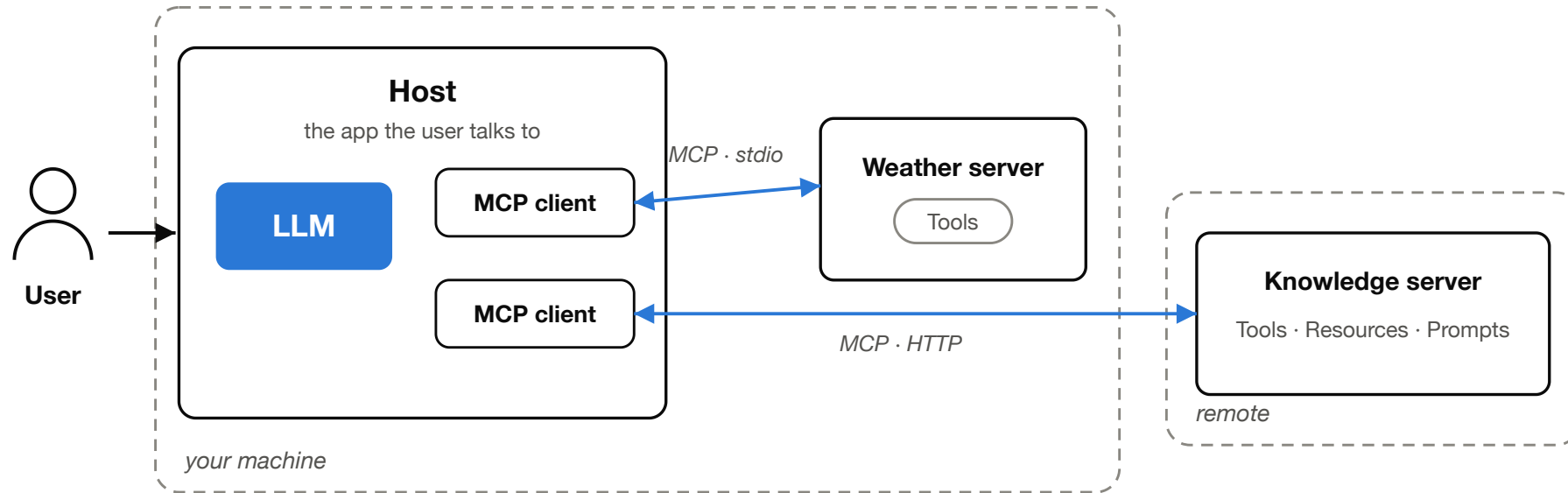
The problem: every app × every tool



2023/24 in practice: ChatGPT plugins · LangChain tools · per-IDE extensions · hand-rolled glue, none of them compatible

N apps × M tools = N × M custom integrations. With one shared protocol: N + M.

How MCP works



- A **server** offers up to three things: **Tools** (functions the model can call) · **Resources** (data for context) · **Prompts** (reusable templates)
- Two transports: **stdio** (local — the server runs on your machine) · **HTTP** (remote — you connect to the provider's server)

Thousands of servers already exist

Check before you build — someone probably built it:

- registry.modelcontextprotocol.io — the official registry
- github.com/mcp · pulesemcp.com · mcpservers.org — large catalogs
- mcpsafe.org — catalog with safety scores
- Vendors increasingly ship an **official server**: GitHub, Atlassian, Stripe, Notion, Asana, ...

First question for any use case: does my software vendor already offer one?

How the model picks a tool: it *reads*

When a server connects, every tool introduces itself in plain text:

```
name:      get_forecast
description: "Daily weather forecast for a city. Use for questions
              about rain, temperature or wind on a specific date."
inputs:     city (text), date (date)
```

The model chooses tools by reading these descriptions — like you choose from a menu.

That has two consequences:

-  well-written descriptions are the biggest quality lever
-  descriptions (and everything a tool returns) are **text that steers your agent** — written by the server's author

! Security: whom are you trusting?

A connected server acts with the permissions you gave it — its code, your credentials, your name.

- **Prompt injection:** the model cannot reliably tell *your* instructions from instructions hidden in text it reads — an email, a web page, a tool description (*OWASP LLM01*)
- **The Lethal Trifecta** (*Willison*) — an agent that combines all three can be made to leak:

 private data	 content from outsiders	 a channel out
customer records, mail, internal docs	inbound email, web pages, tickets	can send / post / publish

From demonstration to reality — in four months

- **May 2025** (*demonstrated*) — **GitHub MCP**: a malicious issue in a public repo steers an agent into copying **private**-repo data into a **public** pull request
- **June 2025** (*an accident*) — **Asana**: the vendor's own hosted MCP server shows one company's project data to another · ~1,000 customer orgs affected
- **Sept 2025** (*in the wild*) — **postmark-mcp**: a fake email server behaves correctly for months, then silently **BCCs every email to its author**

Three different failure modes: a hijacked agent · an honest vendor's bug · a malicious server.



Contain the damage instead of hoping

- **Least privilege** — the agent gets its **own account**, minimal scopes, short-lived keys — never your admin login
- **Human in the loop** — a person approves everything **irreversible**: send, delete, pay, publish
- **Rule of Two (*Meta*)** — per agent, at most **2 of the 3** Trifecta properties; the third only behind an approval
- **Trust the source** — prefer first-party/verified servers · treat server updates like software updates
- **Keep logs** — when something goes wrong, you want to know what the agent did

Mindset: assume the agent *can* be hijacked — and make sure that's survivable.

Your vetting checklist for any MCP server

(it's on your table — you'll use it in a few minutes)

#	Question group	The incident behind it
1	Who provides it? first-party · verified · unknown	postmark-mcp (fake vendor)
2	What can it do in your name? read-only vs. irreversible	GitHub (over-broad token)
3	Where does your data go? local code vs. remote cloud	Asana (vendor cloud bug)
4	Does the combination become dangerous? Trifecta / Rule of Two	GitHub (all three legs)
5	Could you live with the worst case?	all of them

→ Verdict: ● use it · ● use with mitigations · ● not in this context

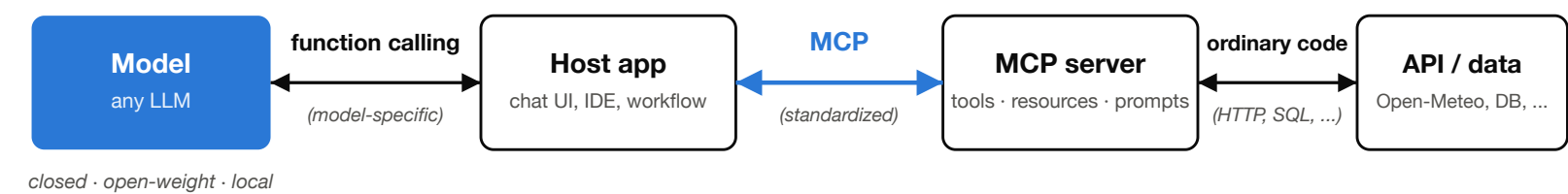
Now you: design your own agentic workflow

In pairs · ~20 minutes · paper and pen

1. Pick a **recurring task** from your company (look things up in 2–3 systems, produce a draft)
2. Fill in the **worksheet**: the idea, example requests, why it needs an agent
3. **Find candidate MCP servers** in the directories (*QR codes on the worksheet — or use the printed server menu*)
4. Run each server through the **vetting checklist** → verdict 
5. Note your **next steps to a pilot**

Afterwards: 2-minute pitches — your idea, your servers, your verdict.

Backup · "Isn't this just another API?"



	A classic API (REST, gRPC, ...)	MCP
Contract	one per service, each different	one fixed interface for every tool
Integration	design time: read docs, write glue code	runtime: <code>tools/list</code> → model context
Consumer	a developer	the model, reading descriptions

Underneath: JSON-RPC, the LSP/ODBC trick · the model still does plain function calling → any model works

Backup · What's actually on the wire

Discovery: the client asks, every server answers the same way

```
→ {"method": "tools/list"}
← {"tools": [{
  "name": "get_forecast",
  "description": "Daily weather forecast for a city. Use for questions
                  about rain, temperature or wind on a specific date.",
  "inputSchema": {"type": "object",
                  "properties": {"city": {"type": "string"},
                                "date": {"type": "string", "format": "date"}},
                  "required": ["city", "date"]}}, ...]}
```

The call, after the model has picked the tool (*JSON-RPC envelope trimmed*)

```
→ {"method": "tools/call",
   "params": {"name": "get_forecast", "arguments": {"city": "Potsdam", "date": "2026-08-01"}}}
← {"content": [{"type": "text", "text": "Sat 2026-08-01, Potsdam: light rain from 14:00, high 19 °C"}]}
```

Plain JSON-RPC · discovery at runtime · the `description` is a prompt